

1 PySpark Overview

- PySpark is the heart of Apache Spark
- PySpark is the Python API for Apache Spark — it enables real-time, large-scale data processing in a distributed environment using Python
- PySpark supports all of Spark's features: Spark SQL, DataFrames, Structured Streaming, Machine Learning (MLlib), pipelines, and Spark Core
- The Pandas API is enabled on top of Spark

Practice Environment

We practice PySpark on **Databricks**.

2 Data Reading

Reading CSV Files

header=True → the first row is treated as the column names

inferSchema=True → Spark automatically detects the data types of the columns

load → refers to the path of the dataset

```
df = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load("/Volumes/pyspark_data/default/pyspark_datasets/gold.dim_customers.csv",)
```

Display the Data

df.show() → shows the first 20 rows of the dataset

display(df) → Databricks-specific function that shows an interactive table

Show Schema of the Dataset

df.printSchema() is a PySpark method that prints the full schema (structure) of the DataFrame. The schema shows: column names, data types, and whether each column contains NULL values (nullable).

```
root
 |-- id: long (nullable = true)
 |-- name: string (nullable = true)
 |-- salary: double (nullable = true)
```

df.schema is a property of a PySpark DataFrame that returns the schema as a **StructType** object instead of printing it.

```
StructType([StructField(&class=class="str">"com">#x27;customer_key', IntegerType(), True),
StructField(&class=class="str">"com">#x27;customer_id', IntegerType(), True),
StructField(&class=class="str">"com">#x27;customer_number', StringType(), True),
StructField(&class=class="str">"com">#x27;first_name', StringType(), True),
StructField(&class=class="str">"com">#x27;last_name', StringType(), True),
StructField(&class=class="str">"com">#x27;country', StringType(), True),
StructField(&class=class="str">"com">#x27;marital_status', StringType(), True),
StructField(&class=class="str">"com">#x27;gender', StringType(), True),
StructField(&class=class="str">"com">#x27;birthdate', DateType(), True),
StructField(&class=class="str">"com">#x27;create_date', DateType(), True)])
```

Read JSON File

```
json_data = spark.read.format(&class=class="str">"com">#x27;JSON')\
.option("inferSchema", True)\
.option("multiline", True)\
.load("/Volumes/pyspark_data/forbes_billionaires_list.json")

display(json_data)
```

Read JDBC (Relational Database) Source

JDBC stands for **Java Database Connectivity** — an API that allows Apache Spark to connect to relational databases such as:

MySQL

PostgreSQL

Microsoft SQL Server

Oracle Database

MariaDB

SQLite (with driver)

Spark uses JDBC to read data from, and write data to, these databases.

```
my_url = "jdbc:postgresql://localhost:5432/mydb"
my_connection = {
    "user": "postgres",
    "password": "postgres",
    "driver": "org.postgresql.Driver"
}

df = spark.read.jdbc(url=my_url, table="orders", properties=my_connection)
```

Read Parquet File

Parquet is a **columnar storage** file format designed for big data processing. It is one of the most common formats used in Apache Spark, Hive, Databricks, and other analytics engines because it is fast and efficient — it stores data column by column.

```
df = spark.read.format("parquet")\
.load("/Volumes/pyspark_data/datasets/customers.parquet")
```

Read JDBC File (alternate syntax)

```
df = spark.read.format("jdbc")\
.option("url", "jdbc:mysql://localhost:3306/pyspark")\
.option("dbtable", "dim_customers")\
.option("user", "root")\
.option("password", "root")\
.load()
```

3 Handling Malformed Records

Means deciding what Spark should do when it encounters data that doesn't match the expected schema (missing fields, extra fields, invalid data types, corrupted records, etc.)

1. PERMISSIVE (default)

- This is the default mode
- Spark keeps all valid rows, sets invalid values to null, and doesn't stop the job
- Most used in production because bad records don't stop the pipeline

```
df = spark.read \
.option("header", True) \
.option("mode", "PERMISSIVE") \
.schema(schema) \
.csv("people.csv")
```

2. DROPMALFORMED

- Simply removes malformed rows
- Use it only if losing bad records is acceptable

```
df = spark.read \
    .option("header", True) \
    .option("mode", "DROPMALFORMED") \
    .schema(schema) \
    .csv("people.csv")
```

3. FAILFAST

- Spark immediately stops when it finds the first malformed record
- Use this when data quality is critical and you don't want to process any invalid data

```
df = spark.read \
    .option("header", True) \
    .option("mode", "FAILFAST") \
    .schema(schema) \
    .csv("people.csv")
```

Saving Bad Records

Instead of losing malformed records, Spark can save them to a dedicated path:

```
df = spark.read \
    .option("mode", "PERMISSIVE") \
    .option("badRecordsPath", "/tmp/badRecords") \
    .schema(schema) \
    .csv("people.csv")
```

Malformed records are written to: `/tmp/badRecords/`

Interview Answer — How does Spark handle malformed records?

Spark provides three read modes: **PERMISSIVE** (default) keeps all rows and replaces invalid values with null; **DROPMALFORMED** skips malformed rows entirely; **FAILFAST** stops processing immediately when a malformed record is found. Additionally, Spark supports `badRecordsPath`, which stores malformed records in a separate location for later inspection while allowing the main job to continue (typically used with PERMISSIVE).

Mode	What happens?	Use case
PERMISSIVE	Invalid fields become null; processing continues	Most production ETL pipelines
DROPMALFORMED	Malformed rows are discarded	When bad records can be safely ignored
FAILFAST	Job fails on the first malformed record	Strict data validation and quality enforcement
badRecordsPath	Saves malformed records to a separate location	Auditing and debugging data quality issues

4 Define Schema for Raw Data

Means explicitly specifying the structure of the raw data before reading it, instead of letting Spark infer the data types automatically. A schema defines:

- Column names
- Data types
- Whether a column can contain NULL values

Why Define a Schema?

- **Faster performance**
 - Spark doesn't need to scan the data to guess data types
- **Better data quality**
 - Ensures columns have the expected data types
 - Detects invalid records early
- **Consistent pipelines**
 - Every time the data is read, the schema remains the same

Define the Schema (StructType)

```
from pyspark.sql.types import *

customer_schema = StructType([
    StructField("customer_id", IntegerType(), False),
    StructField("name", StringType(), True),
    StructField("age", IntegerType(), True),
    StructField("salary", DoubleType(), True)
])
```

Reading the File Using the Schema

```
df = spark.read \
    .schema(customer_schema) \
    .option("header", True) \
    .csv("/FileStore/customers.csv")
```

Summary

Defining a schema for raw data means explicitly specifying the column names, data types, and nullability before reading the data into Spark. This improves performance by avoiding schema inference, ensures consistent data types, and helps detect malformed or invalid records early in the data ingestion pipeline.

You Can Also Write a DDL Schema

```
schema = """
customer_id INT,
name STRING,
age INT,
salary DOUBLE
"""
```

DDL Schema vs. StructType

DDL Schema	StructType
Short and readable	More verbose
Easy for simple schemas	Better for complex schemas
Similar to SQL	Pure PySpark API
Common in Databricks notebooks	Common in production code requiring fine-grained control

5 SELECT Transformation & Alias

The `select` transformation in PySpark is used to choose specific columns from a DataFrame. It can also be used to rename columns, create new columns using expressions, and perform calculations.

```
class=class="str">"com"># Select one or more columns
df.select("name", "salary").show()

class=class="str">"com"># Select all columns
df.select("*").show()
```

Alias

An alias is a temporary name given to a column, DataFrame, or table. It exists only for the current query or operation and does not rename the original object.

```
df.select(col("salary").alias("employee_salary")).show()
```

6 Filter Transformation

- Used to keep only the rows that satisfy a given condition
- It is a **lazy transformation** — Spark doesn't execute it until an action is called
- It is a **narrow transformation**

```
df.filter(df.age > 30).show()

df.filter(col("salary").isNull()).show()

df.filter((df.age < 30) | (df.salary > 7500)).show()

df.filter((df.age > 30) & (df.salary >= 7000)).show()

df.filter(col("department").isin("IT", "HR")).show()
```

7 withColumn(), withColumnRenamed() & lit()

withColumn()

- Used to add a new column or replace an existing column in a DataFrame
- It is a lazy transformation — needs an action to be executed

```
df2 = df.withColumn("bonus", col("salary") * 0.10)

df2 = df.withColumn("salary", col("salary").cast("double"))
```

withColumnRenamed()

- Used to rename an existing column in a DataFrame
- Permanently renames a column in the returned DataFrame
- It is a lazy transformation

```
df2 = df.withColumnRenamed("fname", "first_name")
```

lit()

- A Spark SQL function used to create a column with a constant value
- Commonly used with withColumn() to add the same value to every row in a DataFrame

```
df2 = df.withColumn("country", lit("Egypt"))

df2 = df.withColumn("bonus", lit(500))

df2 = df.withColumn("is_active", lit(True))
```

8 String & Numeric Functions

regexp_replace()

A Spark SQL function used to replace text in a string column using a regular expression (regex).

```
class=class="str">"com"># Remove all hyphens
df2 = df.withColumn("phone", regexp_replace(col("phone"), "-", ""))

class=class="str">"com"># Remove all digits
df2 = df.withColumn("text", regexp_replace(col("text"), "\\d", ""))

class=class="str">"com"># Remove all special characters
df2 = df.withColumn("text", regexp_replace(col("text"), "[^a-zA-Z0-9]", ""))
```

concat_ws()

- Stands for **concatenate with separator**
- Used to join multiple string columns into a single string, inserting a specified separator between the values
- It skips NULL values

```
class=class="str">"com"># Create the full name
df2 = df.withColumn("full_name", concat_ws(" ", "first_name", "last_name"))

class=class="str">"com"># Create the full address
df2 = df.withColumn("full_address", concat_ws(", ", "street", "city", "country"))
```

round()

Used to round numeric values to a specified number of decimal places — commonly used with withColumn() to format prices, averages, percentages, or calculated values.

```
class=class="str">"com"># Round to 2 decimals
df2 = df.withColumn("price", round(col("price"), 2))

class=class="str">"com"># Round to a whole number
df2 = df.withColumn("price", round(col("price"), 0))
```

9 Type Casting Transformation

The process of converting a column from one data type to another — commonly used when data is read from CSV, JSON, or text files because many columns are initially loaded as StringType.

```
class=class="str">"com"># Convert age (string) to integer
df2 = df.withColumn("age", col("age").cast("int"))

class=class="str">"com"># Convert string to double
df2 = df.withColumn("salary", col("salary").cast("double"))

class=class="str">"com"># String to date
df2 = df.withColumn("birth_date", col("birth_date").cast("date"))

class=class="str">"com"># String to timestamp
df2 = df.withColumn("created_at", col("created_at").cast("timestamp"))
```

Common Data Types

Data Type	Description
"string"	Text
"int" / "integer"	Integer
"long"	Large integer
"double"	Decimal number
"float"	Floating-point number
"boolean"	True or False
"date"	Date
"timestamp"	Date and time
"decimal(10,2)"	Fixed-precision decimal

10 Sorting a DataFrame

Sorting a DataFrame arranges the rows in ascending or descending order based on one or more columns. PySpark provides two equivalent methods: `sort()` and `orderBy()`.

```
class=class="str">"com"># Sort the salary in ascending order
df.sort("salary").show()

class=class="str">"com"># Sort the salary in descending order
df.sort(col("salary").desc()).show()

class=class="str">"com"># Sort by multiple columns
df.sort("salary", "name").show()

class=class="str">"com"># Mixed sorting
df.sort(
    col("salary").desc(),
    col("name").asc()
).show()

class=class="str">"com"># Sort using orderBy()
df.orderBy("salary").show()
```

sort() vs. orderBy()

There is no functional difference. Both methods sort a DataFrame and can sort by one or multiple columns in ascending or descending order.

Method	Description
<code>sort("col")</code>	Sort ascending (default)
<code>orderBy("col")</code>	Same as sort()
<code>col("col").asc()</code>	Explicit ascending order
<code>col("col").desc()</code>	Descending order
<code>sort("c1", "c2")</code>	Sort by multiple columns
<code>orderBy(col("c1").desc(), col("c2").asc())</code>	Mixed ascending / descending sorting

11 limit()

Returns the first N rows from a DataFrame. Commonly used to preview a dataset, test queries on a small sample, or reduce the amount of data displayed or processed.

```
class=class="str">"com"># Return the first 3 rows
df.limit(3).show()

class=class="str">"com"># Show the top 3 highest salaries
df.orderBy(col("salary").desc()).limit(3).show()
```

Method	Description
<code>df.limit(5)</code>	Return the first 5 rows
<code>df.orderBy("salary").limit(5)</code>	Return the first 5 rows after sorting
<code>df.orderBy(col("salary").desc()).limit(10)</code>	Return the top 10 highest salaries

12 drop() & dropDuplicates()

drop()

- Used to remove one or more columns from a DataFrame
- It only removes columns — to remove rows, use `filter()`, `when()`, or `dropDuplicates()`

```
class=class="str">"com"># Drop a specific column
df2 = df.drop("age")

class=class="str">"com"># Drop multiple columns
df2 = df.drop("id", "age")
```

dropDuplicates()

- Used to remove duplicate rows from a DataFrame
- Keeps only one occurrence of each duplicate row
- The `subset` parameter removes duplicates based on specific columns

```
class=class="str">"com"># Remove all duplicate rows from the entire DataFrame
df2 = df.dropDuplicates()

class=class="str">"com"># Remove duplicate rows on the "id" column (subset parameter)
df2 = df.dropDuplicates(["id"])
```

Note

When using a subset parameter, PySpark keeps one **arbitrary** row for each unique value in the subset. If you need to keep the latest or highest-valued row, use a **window function** instead.

13 union() & unionByName()

Both are used to combine the rows of two DataFrames (similar to SQL `UNION ALL`). The main difference is how they match columns.

Method	Matches Columns By	Column Order Must Match
<code>union()</code>	Position	Yes
<code>unionByName()</code>	Column Name	No

`union()`

- Combines two DataFrames by column position
- Both DataFrames must have:
 - The same number of columns
 - Compatible data types
 - Columns in the same order
- Doesn't remove duplicates (equivalent to SQL UNION ALL) — use `distinct()` to remove duplicates

```
df3 = df1.union(df2)
```

`unionByName()`

- Combines DataFrames by column names instead of positions
- Column order doesn't matter
- Column names and data types must be compatible
- Doesn't remove duplicates — use `distinct()` to remove duplicates

```
df3 = df1.unionByName(df2)
```

Interview Answer — `union()` vs. `unionByName()`

`union()` matches columns by position, while `unionByName()` matches columns by name.

14 Date & Timestamp Functions

PySpark provides many built-in functions for working with dates and timestamps. These are commonly used in ETL pipelines, reporting, and data analysis.

current_date() & current_timestamp()

```
df.select(current_date().alias("today")).show()

df.select(current_timestamp().alias("now")).show(truncate=False)
```

to_date() & to_timestamp()

```
class=class="str">"com"># Convert string to DateType
df2 = df.withColumn("date", to_date("date"))

class=class="str">"com"># Specify a format
df2 = df.withColumn("date", to_date("date", "dd/MM/yyyy"))

class=class="str">"com"># Convert string to TimestampType
df2 = df.withColumn("datetime", to_timestamp("datetime"))
```

date_format()

```
df.select(date_format(current_date(), "dd/MM/yyyy") \
    .alias("formatted")).show()
```

Common Format Patterns

Pattern	Example
yyyy	2026
MM	08
dd	01
MMM	Aug
EEEE	Saturday
HH	15
mm	30
ss	45

datediff(), date_add() & date_sub()

```
class=class="str">"com"># Number of days between two dates
df.select(
    datediff(
        current_date(),
        to_date(lit("2026-07-01"))
    ).alias("days")
).show()

class=class="str">"com"># Add days to a date
df.select(date_add(current_date(), 10).alias("future")).show()

class=class="str">"com"># Subtract days from a date
df.select(date_sub(current_date(), 10).alias("past")).show()
```

add_months() & months_between()

```

class=class="str">"com"># Add months
df.select(
    add_months(current_date(), 2).alias("after_2_months")
).show()

class=class="str">"com"># Subtract months
df.select(
    add_months(current_date(), -3)
).show()

class=class="str">"com"># Months between two dates
df.select(
    months_between(
        current_date(),
        to_date(lit("2026-01-01"))
    ).alias("months")
).show()

```

Extracting Date Parts

```

df.select(year(current_date()).alias("year")).show()

df.select(month(current_date()).alias("month")).show()

df.select(dayofmonth(current_date()).alias("day")).show()

df.select(dayofweek(current_date()).alias("weekday")).show()

df.select(dayofyear(current_date()).alias("day")).show()

df.select(weekofyear(current_date()).alias("week")).show()

```

last_day() & next_day()

```

class=class="str">"com"># Last day of the month
df.select(
    last_day(current_date()).alias("last_day")
).show()

class=class="str">"com"># First occurrence of the specified weekday after the given date
df.select(
    next_day(current_date(), "Monday").alias("next_monday")
).show()

```

Summary of Date Functions

Function	Description
<code>current_date()</code>	Current date
<code>current_timestamp()</code>	Current date and time
<code>to_date()</code>	Convert string to DateType
<code>to_timestamp()</code>	Convert string to TimestampType
<code>date_format()</code>	Format a date or timestamp
<code>datediff()</code>	Difference in days
<code>date_add()</code>	Add days
<code>date_sub()</code>	Subtract days
<code>add_months()</code>	Add or subtract months
<code>months_between()</code>	Difference in months
<code>year()</code>	Extract year
<code>month()</code>	Extract month
<code>dayofmonth()</code>	Extract day of month
<code>dayofweek()</code>	Extract weekday number
<code>dayofyear()</code>	Extract day of year
<code>weekofyear()</code>	Extract week number
<code>last_day()</code>	Last day of month
<code>next_day()</code>	Next specified weekday